

Greenhouse Management

Analysis and Advice

1. Introduction

This project involves building a greenhouse management prototype for one plant within four weeks. The system will monitor growing conditions, water the plant automatically, and show information on a local OLED display and a remote dashboard. Measurements and watering events will be stored in a database so the user can review what happened over time.

The project uses development-grade hardware to test the behaviour and integration of the system. The priority is a working prototype with dependable irrigation and monitoring.

2. Goals and objectives

The main goal is to build a working greenhouse management prototype that monitors one plant, controls irrigation locally and provides an OLED display, database storage, a dashboard and alerts within four weeks.

SMART criterion	Application to the greenhouse project
Specific	Develop one prototype for one plant, covering environmental monitoring, automatic irrigation, an OLED display, database storage, a dashboard and alerts.
Measurable	The system can read the sensors, show readings on the OLED, store them in the database and display them on the dashboard. It can water dry soil when water is available, stop watering when the reservoir is low and generate the required alerts.
Achievable	I have previous experience with embedded systems programming and I am motivated to learn the new concepts needed for this project. Using available hardware and focusing on one plant keeps the scope manageable.
Relevant	The project will help me improve my embedded C skills, learn ESP-IDF and understand how an embedded device connects to a backend and database. It will also help me become more independent in learning new technologies.
Time bound	Complete and test the prototype within the four-week project period.

Learning objectives

- Improve my embedded C programming skills.
- Learn the ESP-IDF framework.
- Learn to connect an embedded device to messaging, persistent storage and a dashboard.
- Use datasheets and official documentation to make and explain technical decisions.
- Improve my process of designing, testing and documenting an integrated system.

3. Scope

The scope is limited by the four-week development period. The main priority is therefore to create a working system for only one plant. Optional functions will be added after the core system works.

In scope

- One plant, one irrigation zone and one external water reservoir.
- Monitoring soil moisture, air temperature, humidity and reservoir water level.
- Automatic irrigation based on soil moisture and available water.
- Local monitoring of the data.
- Sending measurements and system events to a backend and storing them in a database.
- A dashboard showing current readings, historical trends and watering activity.
- Alerts for low water, abnormal environmental conditions and system faults.
- Light monitoring, storage, display and alerts as Should Have functions.
- Automatic fan control and remote configuration as Could Have functions.

Out of scope

- Automatic heating.
- Automatic humidification or dehumidification.
- Automatic artificial lighting.
- Plant disease detection.
- Weather forecasting.
- AI or machine-learning control.
- Multiple independent greenhouse zones.
- Commercial-scale deployment and production-grade industrial hardware.

4. Requirements

The requirements are listed in priority order using MoSCoW prioritisation. Must Haves define the working prototype. Should Haves are planned additions, Could Haves are optional, and Won't Haves are excluded from this project.

Functional requirements and MoSCoW prioritisation

Functionality	MoSCoW priority
Measure soil moisture.	Must
Measure air temperature.	Must
Measure air humidity.	Must
Monitor reservoir water level to detect low water.	Must
Control irrigation automatically using soil moisture and available water.	Must
Prevent irrigation when the reservoir water level is too low.	Must

Functionality	MoSCoW priority
Show current readings and system status locally.	Must
Transmit measurements and system events to the backend.	Must
Store measurements and watering events in the database.	Must
Show current readings, historical trends and watering activity on a dashboard.	Must
Generate alerts for low water, abnormal environmental conditions and system faults.	Must
Measure ambient light level.	Should
Store and display light readings.	Should
Generate alerts for light readings outside the configured range.	Should
Control a ventilation fan automatically.	Could
Use temperature and/or humidity as input for fan control.	Could
Allow selected thresholds or configuration values to be changed remotely.	Could
Automatic heating.	Won't
Automatic humidification or dehumidification.	Won't
Automatic artificial lighting.	Won't
Plant disease detection.	Won't
Weather forecasting.	Won't
AI or machine-learning control.	Won't
Multiple independent greenhouse zones.	Won't
Commercial-scale deployment and production-grade industrial hardware.	Won't

Technical constraints

The controller will be an ESP32 programmed in C with ESP-IDF. Communication will use MQTT and Mosquitto. The backend will use Node.js, PostgreSQL and Grafana. Vim and Git are the selected development tools.

The available DHT11 and M5Stack pump are the starting hardware choices. The soil-moisture sensor, water-level sensor and OLED module must be confirmed before the final wiring design. The OLED displays current information; persistent storage is provided by the database.

Calibration, watering thresholds, pump run time, measurement intervals and alert ranges will be set during design and prototype testing. They must suit the selected plant and hardware. Detecting low water is sufficient for the core requirement; a continuous reservoir percentage is not required.

5. Success criteria

The project is successful when all Must Have functions work together and the following behaviour can be demonstrated:

- Sensor readings appear on the local screen, are saved in the database and can be viewed on the dashboard with their history.
- Dry soil leads to watering when enough reservoir water is available. Low water and invalid critical sensor readings prevent watering.
- The required alerts identify low water, abnormal environmental conditions and system faults.
- Local monitoring and irrigation continue during a connection failure, and data transmission resumes after reconnection.
- The user can understand the displayed information and recognise when action is needed.

6. Advice

Based on the identified needs and the four-week project scope, the stakeholder is advised to begin with a one-plant prototype that combines automatic irrigation with local and remote monitoring. This provides a manageable setup for evaluating whether the solution supports the intended plant-care tasks before investing in a larger installation.

The stakeholder should provide the required hardware and access to a suitable computer or hosting environment for the backend, database and dashboard. The suitability of existing equipment should be assessed before purchasing a dedicated server. Essential watering decisions should remain on the local controller so that irrigation can continue when the network or backend is unavailable.*

The stakeholder must also arrange regular reservoir refilling and equipment checks. Because the prototype only monitors temperature and humidity, someone must respond when these conditions require attention. Expansion to additional plants or environmental controls should be considered after the prototype has been tested against the success criteria.

8 Technical details

Technology comparisons and the reasons for the selected hardware and software are explained in the separate Technical Details document. The design will describe the wiring, interfaces and control logic. The project plan contains the four-week schedule.